



— INFORME REFLEXIVO DE EVALUACIÓN TÉCNICA Y GRUPAL

REGISTRO DE EXPERIENCIA GRUPAL Y AUTOEVALUACIÓN

Reflexión del Equipo de Desarrollo sobre la Resolución de la Problemática · Proyecto Código Azul

INSTITUCIÓN & JURISDICCIÓN

E.E.S.T. N.º 2 · Prov. de Buenos Aires

EQUIPO Y ROLES ASIGNADOS

A. Martínez (QA/Arq), I. Cardozo (Backend), F. Sarraute (Frontend/PWA), A. Heredia (Push/Telem), M. Silvani (Analista/UML)

CERTAMEN

Olimpiada Nacional ETP 2026 (INET)

PROYECTO

Plataforma Hospitalaria Código Azul (Tiempo Real & Misión Crítica)

01 ¿Cómo se organizaron los tiempos, división de tareas y roles?

Para abordar la consigna en los plazos requeridos, adoptamos una metodología ágil estructurada en cuatro fases secuenciales: 1) *Análisis Funcional, Requerimientos IEEE 830 y Modelado Canónico UML 2.5* (a cargo de **Marcos Silvani**); 2) *Construcción del Núcleo Backend, Persistencia Relacional ACID y Transaccionalidad en PostgreSQL 18* (liderado por **Ivan Cardozo**); 3) *Infraestructura de Tiempo Real, Firebase Admin SDK y Auditoría de Telemetría Push* (desarrollado por **Alex Heredia**); y 4) *Desarrollo de Interfaces Integradas (PWA Móvil de Alarma y Dashboard Hospitalario PC con reactividad y gráficos SVG)* desarrolladas integralmente por **Franco Sarraute**. **Alan Martínez** coordinó la arquitectura global, la matriz de trazabilidad y la suite de 17 pruebas automatizadas de aseguramiento de calidad (QA).

02 ¿Cómo funcionaron como equipo?

El equipo operó bajo una dinámica de alta sinergia colaborativa y comunicación continua. La decisión arquitectónica clave (ADR-01) de implementar *Clean Architecture* con patrón *Feature-First* permitió que el backend, la infraestructura de tiempo real y los clientes web/móvil avanzaran en paralelo sin colisiones en Git (mediante *Conventional Commits*). Establecimos contratos rigurosos de interfaz (JSON REST y eventos Socket.IO como `incidente:nuevo`, `incidente:ack`, `incidente:resuelto` e `incidente:cancelado`), lo que posibilitó realizar pruebas cruzadas y revisiones de pares tempranas. La coordinación de QA garantizó que cada componente fuera validado contra la matriz de requerimientos del SRS antes de consolidar la entrega.

03 ¿Cuáles fueron las principales dificultades para la resolución de la tarea? ¿Pudieron resolverlo? ¿Cómo?

Durante el desarrollo se superaron cuatro retos técnicos de alta complejidad propios de los entornos hospitalarios críticos:

1. Idempotencia y Prevención de Pánico

Implementamos una *Barrera de Idempotencia de 60s* indexada en PostgreSQL que retorna el incidente activo con HTTP 200 sin duplicar registros ni emitir alertas redundantes ante pulsaciones repetidas.

2. Notificación en Doze Mode

Configuramos *Firebase Cloud Messaging (FCM)* con prioridad `'high'`, canal sonoro crítico y telemetría JSONB que despierta los terminales móviles en reposo profundo.

3. Blindaje Médico-Legal Inmutable

Desarrollamos un trigger en PostgreSQL (`trg_audit Immutable`) que rechaza irrevocablemente sentencias `UPDATE` o `DELETE` sobre la tabla de auditoría forense `append-only`.

4. Ciclo Clínico & Gestión de Guardia

Incorporamos el caso de uso de resolución (`EN_ATENCION` → `RESUELTO`) con auditoría de tiempo de respuesta y el desregistro seguro de tokens FCM en `logout` para evitar alertar a personal fuera de turno.

04 Reflexión Final y Aprendizaje Técnico

La resolución de esta problemática representó un hito formativo decisivo. Logramos articular estándares internacionales de ingeniería (IEEE 830, ISO/IEC 25010, Normas APA 7.ª Ed.) con buenas prácticas profesionales de arquitectura limpia, transaccionalidad ACID y despliegue containerizado en Docker. Demostramos que el software confiable, auditable y de latencia ultra-baja es una herramienta vital para optimizar los tiempos de respuesta médica y salvar vidas humanas en el sistema de salud pública.

Alan Martínez
Lider Arq. & QA Lead

Ivan Ismael Cardozo
Backend Core & DB Architect

Franco Sarraute
Frontend & Mobile Lead

Alex Heredia
Push & Telemetría Lead

Marcos Silvani
Systems Analyst & UML Lead